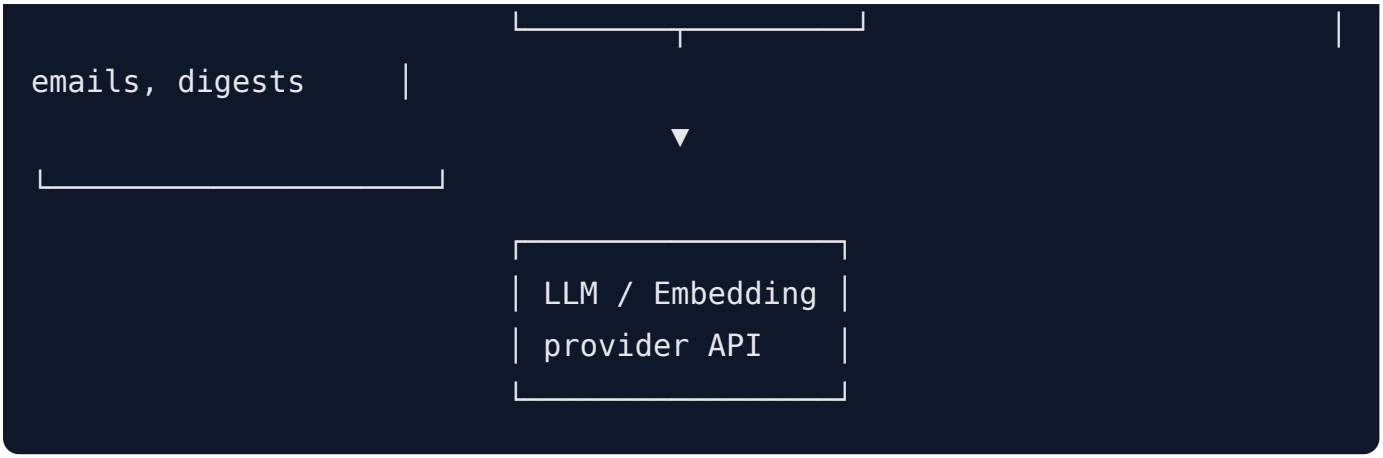
ProjectMatch – Architecture

1. High-Level Overview


HTTPS (REST)
React SPA (Vercel)
Express API
(Render/AWS)
WebSocket (Socket.IO)
- Auth / RBAC
- Domain services
- Socket.IO
gateway
MongoDB Atlas
(primary data)
Redis (optional)
cache, pub/sub,
rate limit, queues
Matching Service
(in-process mod.)
Background Workers
(BullMQ): embeddings



Style: modular monolith. Each domain (auth, users, matching, projects, teams, chat, notifications, tasks, ratings, admin) is a self-contained module with routes → controller → service → model. This keeps deployment simple for a capstone while allowing later extraction of chat or matching into separate services.

---

2. Technology Decisions

Concern	Choice	Rationale
DB	MongoDB	Flexible profile/skill documents, easy embedding storage, fast iteration. Use PostgreSQL + pgvector if you prefer strong relations and SQL analytics.
Auth	JWT access + refresh	Stateless API, works with Socket.IO handshake
Real-time	Socket.IO	Rooms, reconnection, fallbacks, Redis adapter for scaling
AI	Hybrid	Embeddings for semantic similarity, rules for complementarity and availability, LLM only for explanations (cheap, deterministic ranking)
Validation	Zod / Joi	Request validation at the boundary

---

3. Data Model (MongoDB)

User

```
_id, name, email (unique), passwordHash, role: student|admin,
isVerified, isActive, college, year, branch, avatarUrl, bio,
skills: [{ skill: ObjectId→Skill, level: 1-5, yearsExp }],
interests: [String],
preferredTech: [ObjectId→Skill],
experience: [{ title, description, link }],
availability: { hoursPerWeek, timezone, slots: [String] },
lookingFor: [hackathon|project|internship|competition],
embedding: [Number],        // vector of profile text
reputation: { avg, count },
createdAt, updatedAt
```

Skill

```
_id, name (unique), category (frontend|backend|ml|design|devops|...),
aliases: [String], isActive
```

Project (project / hackathon / internship / competition listing)

```
_id, title, description, type, owner: ObjectId→User,
requiredRoles: [{ role, skills: [ObjectId→Skill], slots, filled }],
requiredSkills: [ObjectId→Skill], tags, deadline, startDate,
mode: online|offline|hybrid, teamSize: { min, max },
status: open|in_progress|completed|closed|flagged,
embedding: [Number], createdAt, updatedAt
```

Team

```
_id, project: ObjectId→Project, name,
members: [{ user, role, joinedAt }], leader: ObjectId→User,
status: forming|active|completed|disbanded
```

Request (invitations & applications)

```
_id, type: invite|application, project, team, from: User, to: User,
role, message, status: pending|accepted|declined|cancelled|expired,
createdAt
```

Message / Conversation

```
Conversation: _id, type: team|direct, team?, participants: [User],
lastMessageAt
Message: _id, conversation, sender, text, attachments: [{url,type}],
         readBy: [User], createdAt
```

Notification

```
_id, user, type, title, body, data: {...}, isRead, createdAt
```

Task

```
_id, team, title, description, assignee, createdBy,
status: todo|in_progress|review|done, priority: low|med|high,
dueDate, labels, comments: [{ user, text, createdAt }], order
```

Rating

```
_id, project, team, rater, ratee,
scores: { teamwork, communication, skill, reliability } (1-5),
comment, createdAt   // unique(project, rater, ratee)
```

Report (admin moderation)

```
_id, reporter, targetType: user|project|message, targetId, reason,
status, resolvedBy
```

Key indexes

- `User.email` (unique), `User.skills.skill`, text index on `name`, `bio`

- `Project.status + deadline`, `Project.requiredSkills`, text index on `title`, `description`
- `Message.conversation + createdAt`
- `Notification.user + isRead + createdAt`
- `Task.team + status`
- Vector search index on `User.embedding` and `Project.embedding` (MongoDB Atlas Vector Search, or in-memory cosine for small datasets)

---

4. AI Team Matching Pipeline

Goal: recommend teammates whose skills **complement** the seeker's or project's needs, not just resemble them.

4.1 Inputs

- Seeker profile, or a Project with required roles/skills
- Candidate pool: active, verified users with matching `lookingFor`, excluding existing members and users with pending requests

4.2 Stage 1 – Candidate retrieval (semantic)

1. Build profile text: `skills + levels + interests + experience + preferred tech`.
2. Generate embedding (cached; refreshed by a background job when the profile changes).
3. Retrieve top-N (e.g., 100) candidates by cosine similarity to the project/seeker embedding.

4.3 Stage 2 – Rule-based scoring (re-rank)

```
finalScore =  0.35 * semanticSimilarity
            + 0.30 * skillCoverage          // required skills the
candidate fills
            + 0.15 * complementarity        // fills gaps the current
team lacks
            + 0.10 * availabilityOverlap
            + 0.05 * reputation
            + 0.05 * interestOverlap
```

- **skillCoverage** = Σ(level-weighted matched required skills) / Σ(required skills)
- **complementarity** = share of the *currently uncovered* skills the candidate covers (for team-building) or `1 – Jaccard(seekerSkills, candidateSkills)` constrained by relevance (for peer matching)
- **availabilityOverlap** = overlap of hours/week and time-zone slots
- Weights live in config so they can be tuned or learned from accept/decline feedback.

4.4 Stage 3 – LLM explanation (top-K only)

Send the top 5–10 results (structured JSON, no sensitive data) to the LLM to produce a short "Why this match" paragraph and a suggested role. Explanations are cached per (seeker, candidate, profile-version). The LLM never changes the ranking, which keeps results reproducible and cheap.

4.5 Feedback loop

Log impressions, invites sent, accepted/declined. Use these for weight tuning and offline evaluation (precision@k, acceptance rate).

4.6 Cold start & fallbacks

- No embedding yet → skill/tag overlap scoring only.
- LLM down → show rule-based "matched skills" chips instead of narrative.

---

5. Real-Time Design (Socket.IO)

- **Auth:** JWT passed in the handshake (`auth.token`), verified in middleware; socket joins `user:<id>` room.
- **Rooms:** `user:<id>` (notifications), `conversation:<id>` (chat), `team:<id>` (task updates).
- **Events**

Event	Direction	Payload
`chat:join` / `chat:leave`	C→S	conversationId
`chat:message`	C↔S	{ conversationId, text, attachments }
`chat:typing`	C↔S	{ conversationId, isTyping }

`chat:read`	C→S	{ conversationId, messageIds }
`notification:new`	S→C	notification object
`task:created/updated/moved`	S→C	task object
`presence:update`	S→C	{ userId, online }

- **Persistence first:** messages are saved to DB, then emitted (ack to sender with the saved id).
- **Scaling:** `@socket.io/redis-adapter` for multiple instances; sticky sessions on the load balancer.

---

6. Core Flows

Invitation flow

1. Leader views AI recommendations → clicks *Invite* → `POST /requests` (type=invite).
2. Notification created and pushed in real time (+ email if offline).
3. Invitee accepts → server validates slot availability in a transaction → adds member, increments `filled`, creates/joins team conversation → notifies leader.

Application flow: reverse of the above (student applies to a project role; owner accepts).

Project completion: owner marks completed → rating window opens → members submit peer ratings → reputation recalculated.

---

7. API Design

- REST under `/api/v1`, JSON, consistent envelope: `{ success, data, error: { code, message, details } }`
- Pagination via `?page=&limit=`; filtering via query params
- RBAC middleware: `student`, `team-leader` (resource-level), `admin`
- Detailed endpoint list in `implementation.md`

---

8. Security

Area	Measures
Passwords	bcrypt (cost ⩾ 12)
Tokens	Access 15 min in memory; refresh 7 d in httpOnly, Secure, SameSite cookie; rotation + revocation list
Input	Schema validation, `express-mongo-sanitize`, `xss-clean`/output encoding
HTTP	`helmet`, strict CORS (client origin only), HTTPS only
Abuse	`express-rate-limit` (stricter on auth, AI endpoints), account lockout after repeated failures
AuthZ	Ownership checks on every mutation (only team members read chat/tasks)
Privacy	Hide email/phone by default; send only necessary fields to the LLM; allow profile visibility toggles
Uploads	Type/size limits, signed URLs to object storage
Secrets	Env variables only; rotate regularly

---

9. Performance & Scalability

- Cache embeddings and recommendation results (Redis, TTL 1 h; invalidate on profile change)
- Cursor-based pagination for chat history
- Background jobs for embeddings, emails, digest notifications
- Lean queries + projections; compound indexes
- Horizontal scaling: stateless API + Redis adapter

---

10. Observability

- Structured logging (`pino`) with request ids
- Error tracking (Sentry)
- Health check `GET /health`, uptime monitor
- Metrics: match acceptance rate, API latency, LLM cost per day

11. Deployment

```
GitHub ──► CI (lint, test, build)
           ├─► Vercel      : client (env: VITE_API_URL,
VITE_SOCKET_URL)
           └─► Render/AWS  : server (Docker or Node service)
                             MongoDB Atlas, Redis (Upstash/Elasticache)
```

- Environments: `development`, `staging`, `production`
- Render: web service with a WebSocket-capable plan, health check path `/health`
- AWS option: ECS Fargate / Elastic Beanstalk behind ALB (sticky sessions), S3 for uploads, CloudFront for static assets
- Database backups enabled (Atlas), migrations/seed scripts versioned

---

12. Admin Dashboard

- KPIs: users, active teams, projects by status, match acceptance rate
- Moderation: review reports, deactivate users, flag/remove projects
- Taxonomy: CRUD for skills/categories
- Audit log for all admin actions

---

13. Testing Strategy

Level	Tools	Focus
Unit	Jest	scoring functions, services, validators
API	Supertest + mongodb-memory-server	auth, permissions, flows
Socket	socket.io-client in Jest	message delivery, room isolation
E2E	Playwright / Cypress	register → profile → match → invite → chat

AI eval	Offline script	precision@k on seeded profiles